Autoscaling and Database Connections

Day 08 | KUBERNETES / EKS / AKS / GKE

Use this guide to investigate a specific failure, apply a cause-specific change and verify recovery. Start with evidence from the affected workload. The examples require adaptation to your environment.

How the failure happens

A replica count is also a multiplier for resources outside Kubernetes. Each application process may own a connection pool. Count every process and rollout surge, not only the steady-state HPA maximum, when estimating the potential database demand.

The Scenario An API scales from 5 to 20 Pods. Each Pod permits 30 database connections.

The Multiplier 20 x 30 = up to 600 app connections, before other clients and rollout surge.

The Bottleneck More app replicas can exceed the database connection budget.

Before you start

Measured database connection capacity and reserve; actual pool settings; process count per Pod; HPA and rollout settings; database session metrics.

Diagnostic checks

```
SHOW max_connections;
SELECT count(*) FROM pg_stat_activity;

# Also inspect pool limits and HPA replicas
```

Check Actual Use: Configured maximum is a ceiling, not proof all connections are open.

Count Everything: Include workers, sidecars, background processes, admin reserve and surge Pods.

Apply the cause specific fix

Fix The Confirmed Cause

```
# Application-specific example, not universal keys
config:
  pool_max_per_process: 10
  maximum_app_processes: 20
budget:
  application_connections: 200
  reserve_for_other_clients: 50
```

Budget First: Set pool limits and replica bounds from database capacity. The keys shown must map to your actual app settings.

Other Options: Evaluate a compatible pooler, shorter transactions, leak fixes and backpressure before scaling blindly.

Validate the result

```
# Load-test through scale-out and rollout.
# Check DB sessions, pool waits and latency.
# Confirm no connection exhaustion.
```

Expected: Traffic increases without exhausting DB connections or hiding queueing failures.

Watch The Trade-Off: Smaller pools can increase waiting. Validate throughput and timeouts.

Practice exercise

Load-test through scale-out and a rolling release. Record session count, pool wait time, query latency and errors. Compare changes with the estimated connection budget.

Implementation notes

The arithmetic assumes one process per Pod for the initial scenario. Each process can own a pool; maxReplicas is not the entire fleet ceiling during rollouts or other workloads. Example keys are a capacity planning document, not a framework configuration. PostgreSQL max_connections is not a recommendation to simply increase it. Pooler mode and application features must be compatible.

Interview question

Why did increasing replicas reduce availability? More processes can overwhelm a shared downstream limit even when application CPU utilization improves.

Record what changed

- The symptom and affected user operation.
- Evidence supporting the root cause and alternatives ruled out.
- The changed configuration and the version or identity involved.
- Positive and negative tests, plus any remaining risks.

Example status

Examples are educational templates or fragments. They were not deployed against a live cloud account or production cluster. A successful syntax check is not proof of runtime compatibility. Test adapted configurations in an isolated environment first.

Official references

Official documentation 1

Official documentation 2